



TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
TRUNG TÂM PHÁT TRIỂN CÔNG NGHỆ THÔNG TIN

BẢO VỆ ĐỒ ÁN TỐT NGHIỆP

NGÀNH CÔNG NGHỆ THÔNG TIN

HỆ ĐÀO TẠO TỪ XA

THÀNH PHỐ HỒ CHÍ MINH, NGÀY 10 THÁNG 10 NĂM 2026

LỄ BẢO VỆ ĐỒ ÁN TỐT NGHIỆP

NGÀNH CÔNG NGHỆ THÔNG TIN - 7480201

TÊN ĐỀ TÀI:

HỆ THỐNG DỊCH GIỌNG NÓI ĐA NGÔN NGỮ GẦN THỜI GIAN THỰC
SỬ DỤNG MÔ HÌNH AI CHẠY CỤC BỘ

A Near Real-Time Speech Translation System Using Locally Hosted AI Models

GVHD:

ThS. Nguyễn Thành Luân

NGƯỜI THỰC HIỆN:

24410300 - Ngô Mạnh Hùng

Thành phố Hồ Chí Minh, ngày 10 tháng 10 năm 2026

Nội dung

I. Tổng quan đề tài

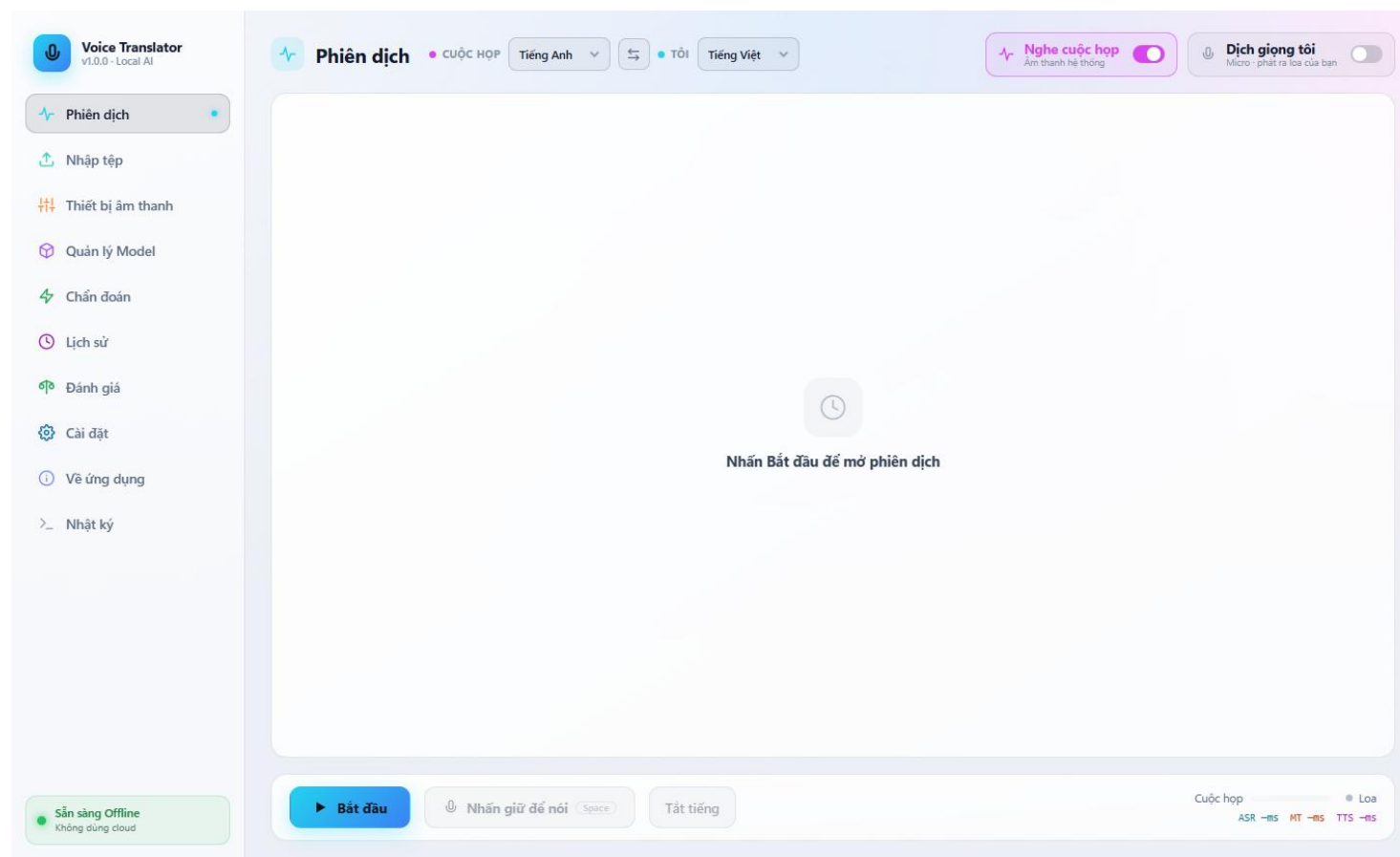
1. Bài toán và các giải pháp liên quan
2. Mục tiêu và phạm vi

II. Phương pháp thực hiện

1. Chuỗi xử lý và lựa chọn mô hình
2. Kiến trúc hệ thống
3. Các kỹ thuật xử lý chính
4. Ứng dụng, đóng gói và kiểm thử

III. Kết quả thực nghiệm

IV. Kết luận



Màn hình Phiên dịch của ứng dụng

I. Tổng quan đề tài

1. Bài toán

Họp và học trực tuyến với người nói ngôn ngữ khác ngày càng phổ biến (Google Meet, Teams, Zoom). Các công cụ dịch giọng nói hiện có gần như đều chạy trên đám mây, kéo theo ba vấn đề:



Phụ thuộc đường truyền

Mạng chậm chờn thì bản dịch trễ hoặc mất giữa cuộc họp.



Tốn phí theo lượt dùng

Tính theo thời lượng hoặc gói thuê bao; nhiều tính năng chỉ có ở gói trả phí.



Nội dung rời khỏi máy

Toàn bộ âm thanh cuộc họp phải gửi tới máy chủ của bên thứ ba.

Trong khi đó, mô hình mã nguồn mở cho từng khâu đã đủ tốt để chạy trên máy cá nhân: **Whisper** nhận dạng ~100 ngôn ngữ [19], **NLLB-200** dịch trực tiếp 200 ngôn ngữ [4], **sherpa-onnx** tổng hợp giọng nói offline [12].



Đề tài trả lời: laptop cá nhân có tự chạy trọn chuỗi dịch giọng nói và theo kịp người nói không?

I. Tổng quan đề tài

1. Các giải pháp liên quan

Tiêu chí	Google Meet [7]	SeamlessM4T [23]	TranscriptionSuite [10]	LocalVocal [22]	Đề tài
Chạy cục bộ, không cần mạng	Không	Có	Có	Có ¹	Có
Nhận dạng tiếng Việt	Có	Có	Có	Có	Có
Dịch có tiếng Việt	Phụ đề, gói trả phí	Có	Chỉ sang tiếng Anh	Có (NLLB, M2M-100)	Có
Đọc bản dịch bằng giọng nói	Có, chưa có tiếng Việt	Có, có tiếng Việt	Không	Không	Có
Thu âm thanh trên máy	Trong cuộc gọi Meet	Không	Micro, âm thanh hệ thống	Mọi nguồn âm thanh của OBS	Micro + hệ thống
Dạng sản phẩm	Tính năng của Meet	Mô hình, dòng lệnh, demo	App Electron + server Docker	Plugin OBS	App desktop

¹ Dịch qua đám mây (DeepL, OpenAI, ...) là tùy chọn. SeamlessM4T chỉ có bản dựng sẵn cho Linux x86-64 và Mac Apple Silicon. Nguồn: trang trợ giúp Google Meet, README của từng dự án, tra ngày 08/10/2026.

➡ Chưa công cụ nào đủ cả sáu hàng: SeamlessM4T gần nhất về mô hình, LocalVocal về ứng dụng

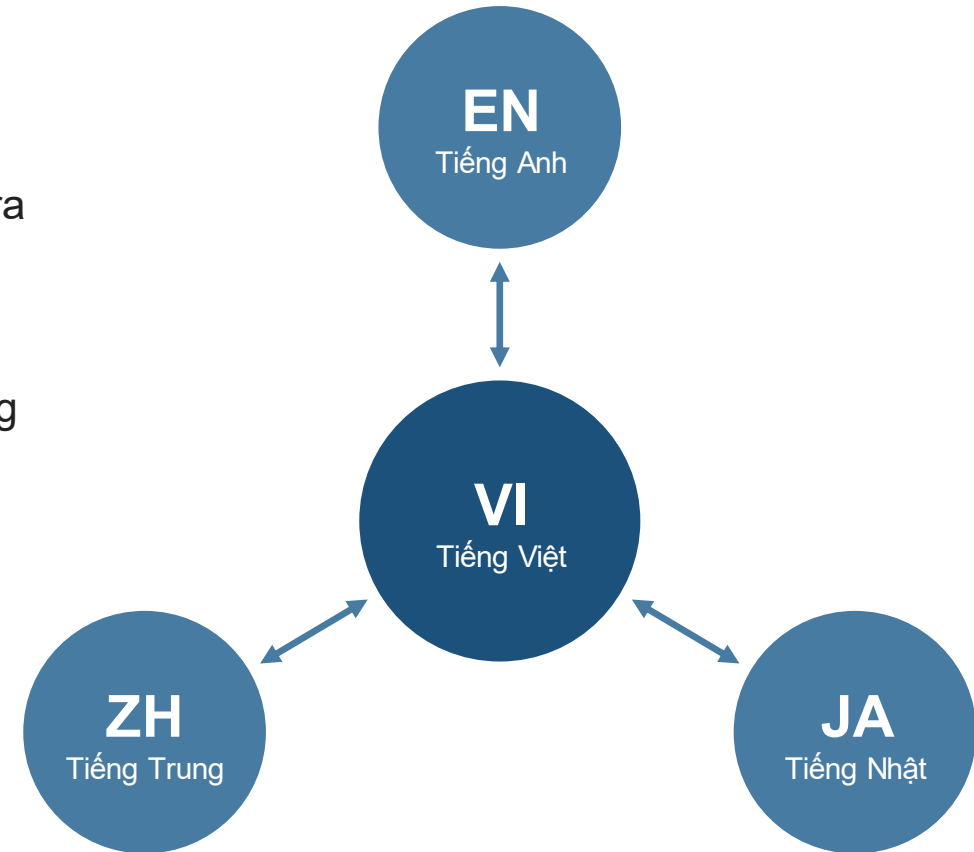
I. Tổng quan đề tài

2. Mục tiêu và phạm vi

- **Input:** giọng người dùng qua microphone và âm thanh đang phát trên máy (phía bên kia cuộc gọi, video, bài giảng)
- **Output:** phụ đề song ngữ trên màn hình; giọng đọc bản dịch ra loa cho người đối diện
- **Ngôn ngữ:** 6 chiều dịch Việt ↔ Anh / Nhật / Trung giản thể
- **Nền tảng:** Windows 11 x64 và macOS 13+ Apple Silicon, đóng gói thành bộ cài
- **Không dùng đám mây** khi dịch; mô hình chỉ tải về một lần

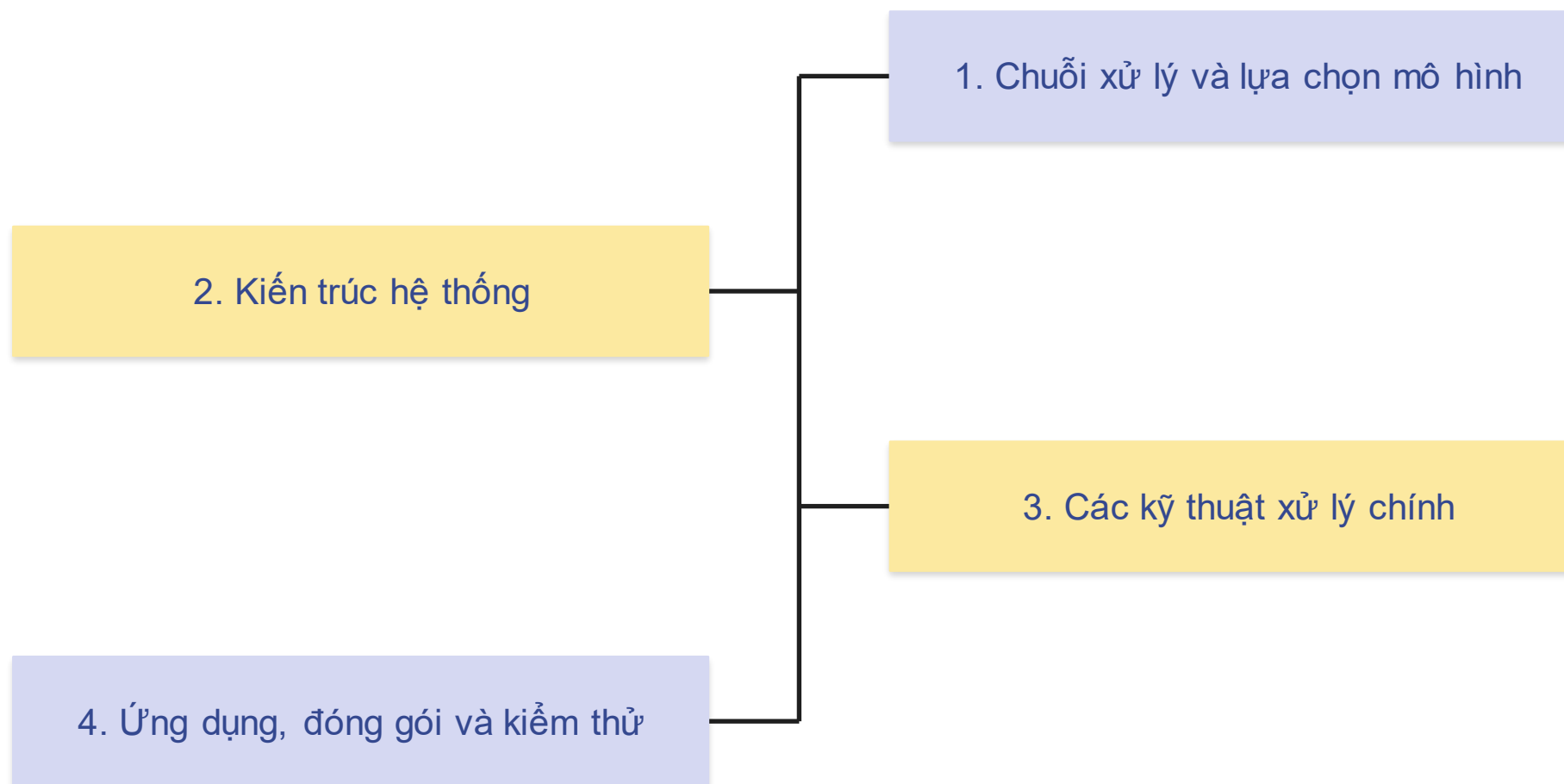
Ngoài phạm vi: không đề xuất mô hình AI mới (đề tài là tích hợp hệ thống); không dịch đồng thời theo từng từ, đơn vị xử lý là một câu sau khi người nói ngắt; không nhận bản giọng; không đưa bản dịch ngược vào phần mềm hợp.

➔ **Mục tiêu: dịch 6 chiều ngay trên laptop, từng câu một, âm thanh không rời khỏi máy**



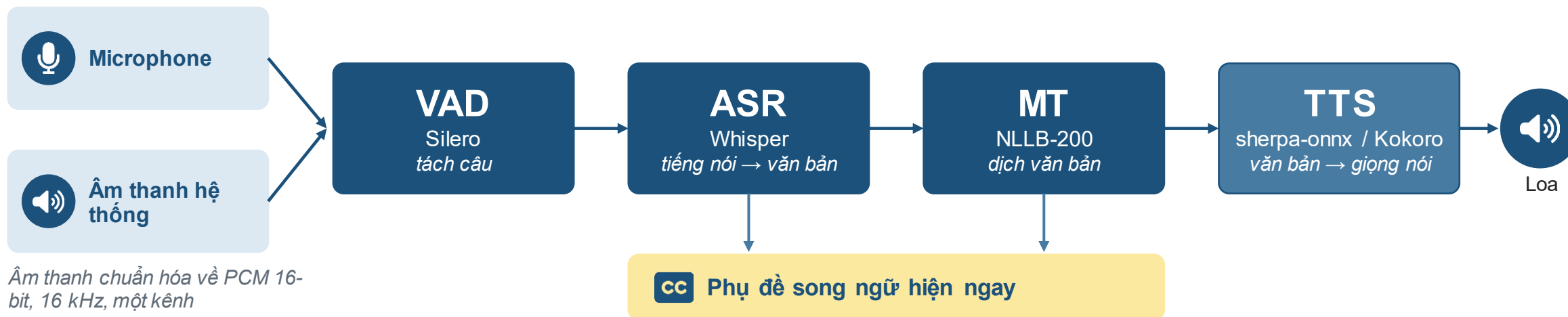
6 chiều dịch, tiếng Việt là trục

II. Phương pháp thực hiện



II. Phương pháp thực hiện

1. Chuỗi xử lý dịch giọng nói



Vì sao chọn cách chuỗi (thay vì đầu-cuối)

- Đo được lỗi từng khâu: nghe nhầm hay dịch nhầm
- Thay từng khâu theo phần cứng, không đụng khâu khác
- Mỗi khâu đã có mô hình mở phủ đủ 4 ngôn ngữ

Cái giá phải trả

- Lỗi cộng dồn: từ nghe nhầm được dịch luôn cái sai
- Độ trễ cộng dồn: bốn khâu chạy nối tiếp
- Cả hai được đo cụ thể ở phần III

➡ **Tách 4 khâu để đo và thay được từng khâu; đổi lại lỗi và độ trễ cộng dồn, nên em đo cả hai**

II. Phương pháp thực hiện

2. Kiến trúc: hai tiến trình trên một máy



Dịch vụ chỉ mở cổng trong máy (127.0.0.1): không ra mạng LAN, không gọi dịch vụ đám mây khi dịch

Vì sao tách hai tiến trình:

Thư viện mô hình (whisper.cpp, transformers, sherpa-onnx) nằm ở Python; Electron thu sẵn được âm thanh hệ thống

Mô hình chiếm ~2 GB RAM và chặn CPU hàng giây: chung tiến trình thì giao diện đơ mỗi câu

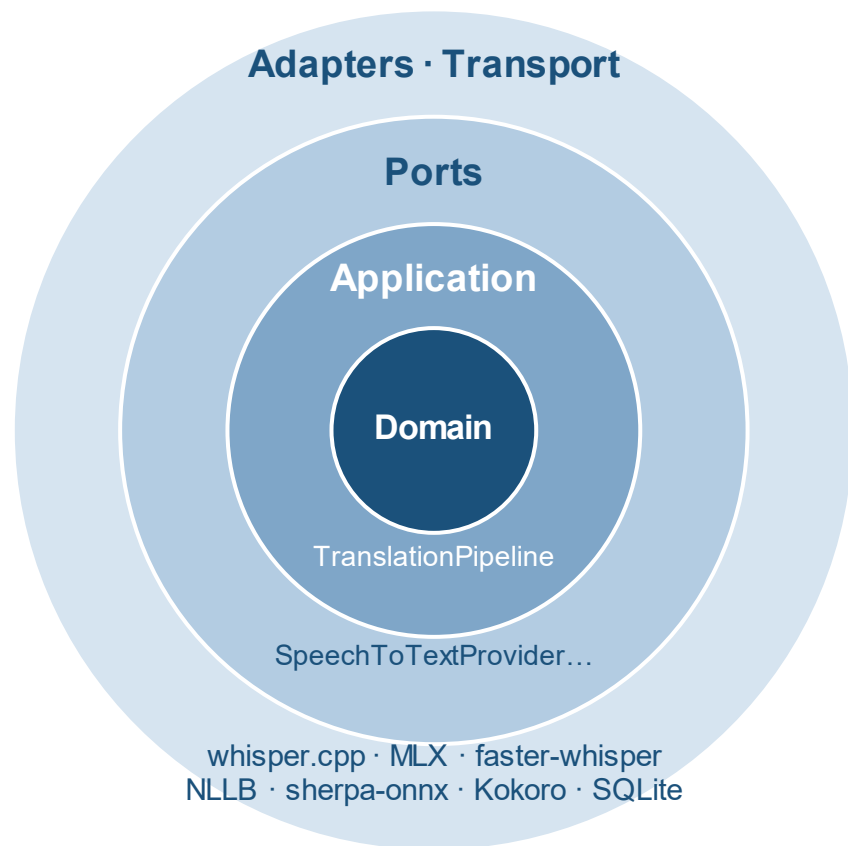
Dịch vụ lỗi thì cửa sổ vẫn mở để báo lỗi và khởi động lại dịch vụ



Tách hai tiến trình để mô hình nặng không làm đơ giao diện; hai bên chỉ trao đổi bên trong máy

II. Phương pháp thực hiện

2. Kiến trúc lục giác (Ports & Adapters) [2]



Một quy tắc: phụ thuộc luôn hướng vào trong

Chuỗi xử lý chỉ biết các port (giao diện trừu tượng), không biết mô hình nào đang chạy phía sau.



Thêm 2 runtime ASR (MLX, faster-whisper)

Mỗi runtime: 1 adapter + 1 dòng đăng ký + 1 dòng preset. Không sửa dòng nào của chuỗi xử lý.



Khi sherpa-onnx không đọc được tiếng Nhật

Thêm adapter Kokoro và LanguageRoutedTts chọn engine theo ngôn ngữ đích; chuỗi xử lý vẫn chỉ thấy một TTS.



Kiểm thử không cần mô hình thật

Thay adapter bằng mô hình giả: 327 bài kiểm thử dịch vụ chạy trong vài giây, tắt định.



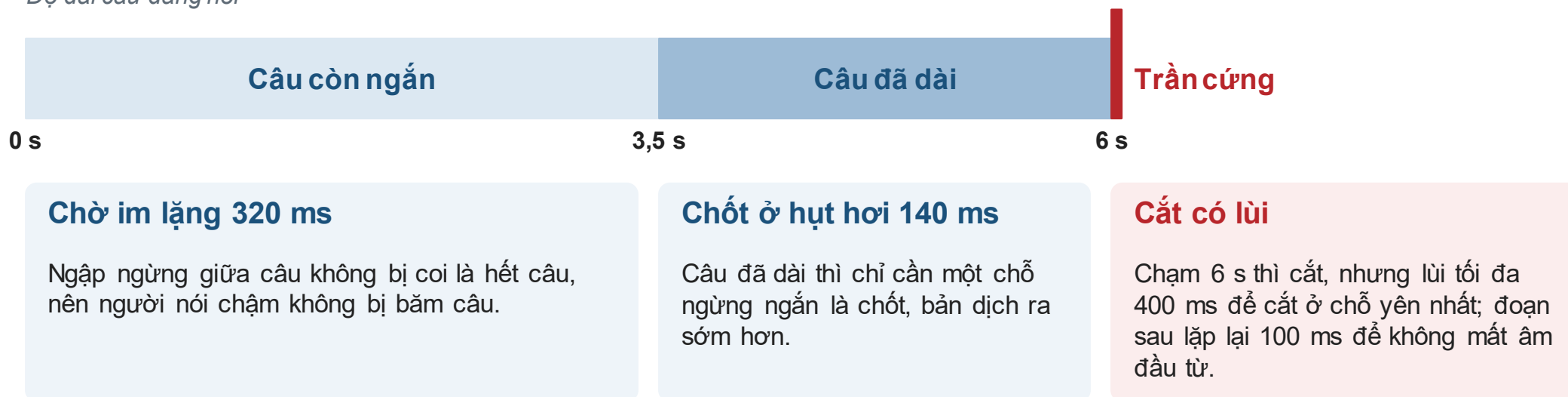
Chuỗi xử lý chỉ biết port, nên thêm 2 runtime ASR và 1 engine TTS mà không sửa dòng nào của nó

II. Phương pháp thực hiện

3. Bộ tách câu: khi nào một câu kết thúc?

Vấn đề: lời nói không có dấu chấm câu. Phiên bản đầu dùng một ngưỡng im lặng 300 ms và trần 20 s: người nói liền mạch phải chờ tới 20 s mới có bản dịch (GVHD góp ý, 19/08/2026).

Độ dài câu đang nói

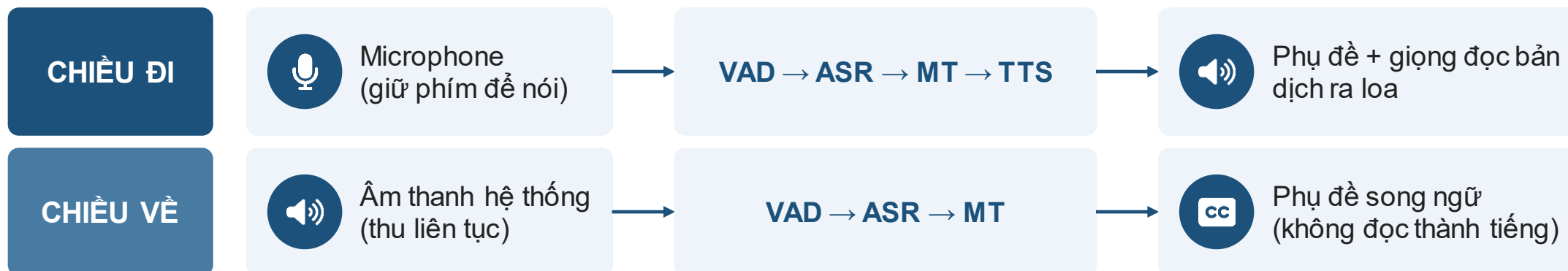


Cái giá: trần 6 s cắt giữa câu 30% số đoạn ở bản tin, 10% ở phỏng vấn, 0% ở TEDx. Các ngưỡng trên là của preset Cân bằng.

➡ **Câu dài nhất giờ là 6 s thay vì 20 s; với bản tin đọc liền mạch, 90% câu ngắn hơn 5,9 s**

II. Phương pháp thực hiện

3. Hai chiều dịch và chống vòng lặp âm thanh



Chống vòng lặp

Giọng đọc bản dịch cũng là âm thanh của máy, nên chiều về sẽ thu lại rồi dịch tiếp. Trong lúc đọc, ứng dụng bỏ qua âm thanh hệ thống và hiện "Tạm ngưng thu".



Nhả phím là chốt câu

Nhả phím thì ứng dụng ngừng gửi âm thanh, nên VAD không thấy khoảng lặng nào để chốt câu. Em thêm `VadStream.flush()` vào port VAD để chốt ngay lúc nhả phím.



Chặn hai lần

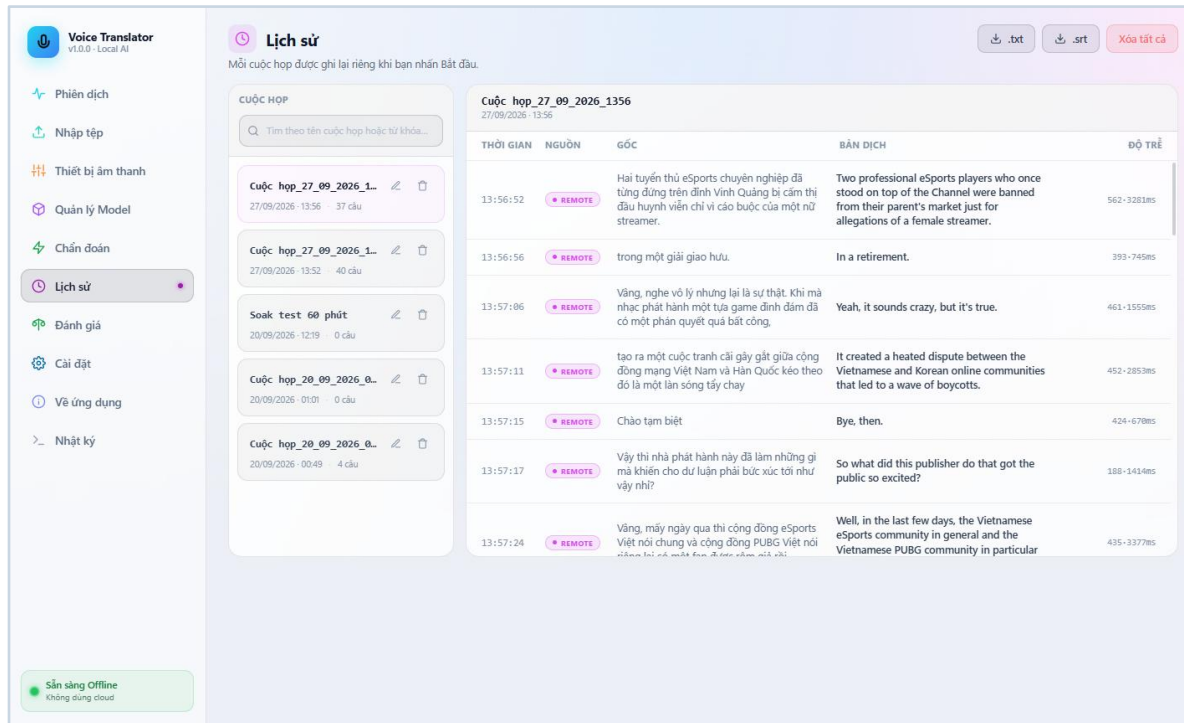
Ngoài lúc giữ phím, ứng dụng không gửi âm thanh micro, và dịch vụ cũng tự bỏ khung micro gửi đến. Một phía sai thì phía kia vẫn chặn.



Hai chiều chạy cùng lúc mà bản dịch đang phát không bị thu lại rồi dịch tiếp

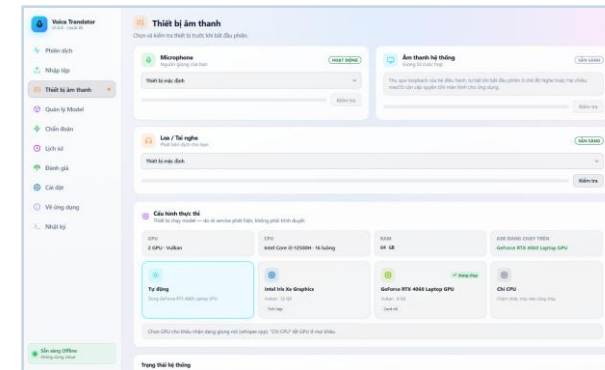
II. Phương pháp thực hiện

4. Ứng dụng desktop

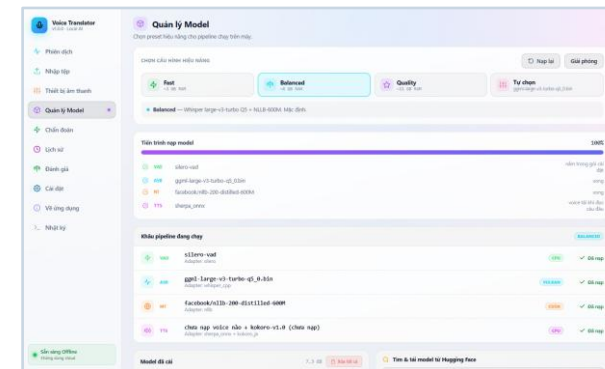


Lịch sử: bản gốc và bản dịch của cả hai chiều

8 màn hình: Phiên dịch · Nhập tệp · Thiết bị âm thanh · Quản lý Model · Chẩn đoán · Lịch sử · Đánh giá · Cài đặt



Thiết bị âm thanh và GPU



Quản lý mô hình, preset và tiến độ nạp



Ngoài phiên dịch trực tiếp, ứng dụng còn nhập tệp, lưu lịch sử, chấm điểm và quản lý mô hình

II. Phương pháp thực hiện

4. Đóng gói và kiểm thử

Bộ cài tự chứa: máy người dùng không cần Python, Node hay trình quản lý gói

.exe 371 MB

Windows 11 x64

.dmg 592 MB

macOS Apple Silicon

Kiểm tra bộ cài Windows theo bốn tầng, từ trong ra ngoài:

Tầng	Kiểm gì	Kết quả
1	Bộ Python đóng gói	Nạp PyTorch, sherpa-onnx, pywhispercpp
2	Dịch vụ chạy từ bộ đó	Nạp 4 khâu trong 34 s, ASR báo Vulkan
3	Bộ cài .exe thật	Dịch vụ tự lên sau 12 s, tắt thì nhả cổng
4	Tốc độ trên bản cài	ASR 3 s âm thanh: 17.573 → 437 ms



Một bộ cài là chạy, không cần cài Python hay Node; 407 bài kiểm thử tự động ở ba lớp

Ba lớp kiểm thử tự động

327

pytest

Dịch vụ: pipeline, tách câu, lọc câu bịa, API, lịch sử

69

vitest + jsdom

Giao diện trên dịch vụ giả

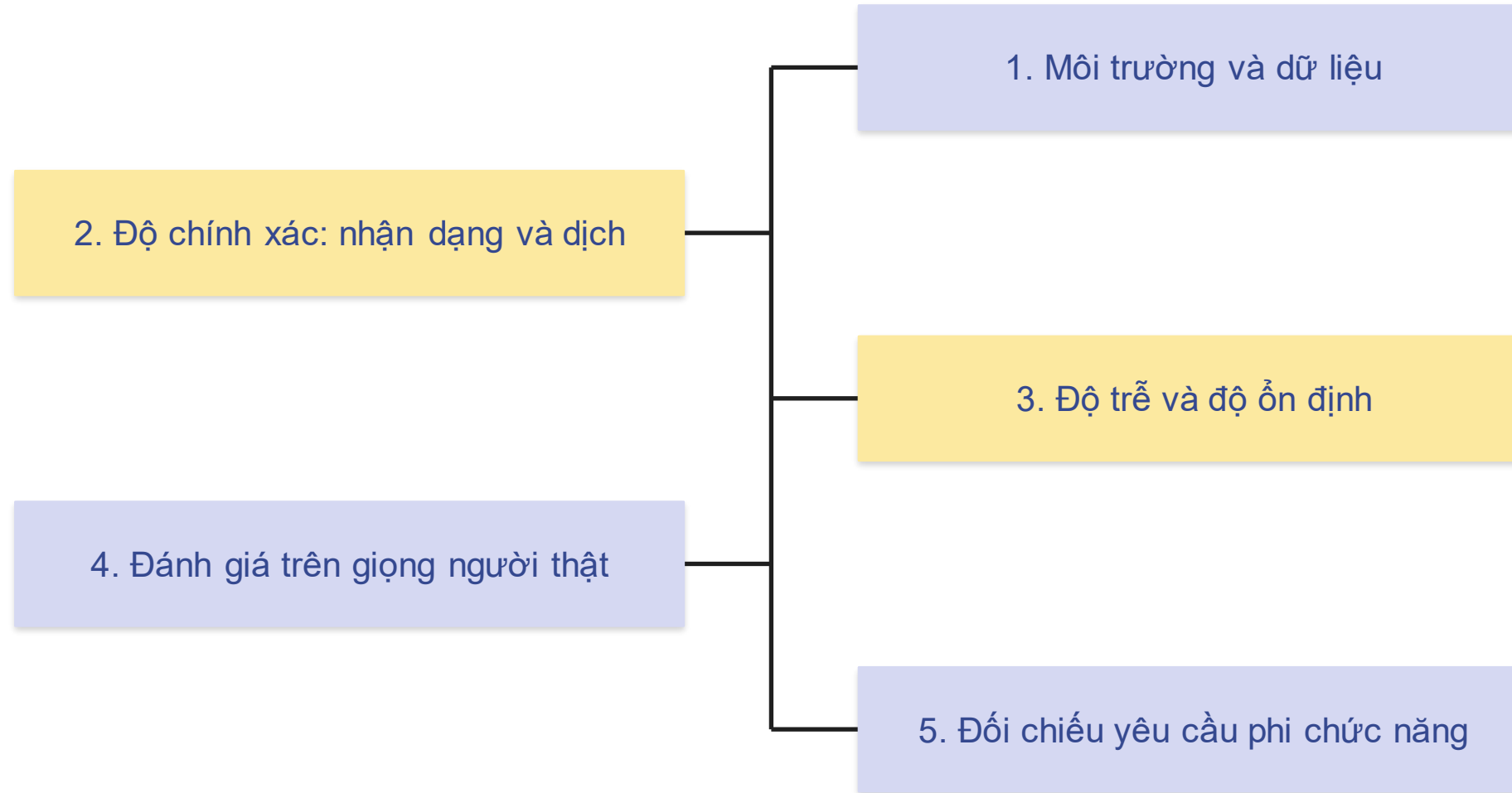
11

Playwright

App Electron đã dựng + dịch vụ thật + tải mô hình thật

Hai lớp đầu dùng mô hình giả: chạy vài giây, tắt định, không cần mạng.

III. Kết quả thực nghiệm



III. Kết quả thực nghiệm

1. Môi trường và dữ liệu thử nghiệm



macOS

Apple M4 · macOS 26.6

ASR: MLX/Metal · MT: PyTorch/MPS · TTS: CPU



Windows

i5-12500H · RTX 4060 8 GB + Iris Xe · 64 GB RAM

ASR: whisper.cpp/Vulkan · MT: PyTorch/CUDA · TTS: CPU

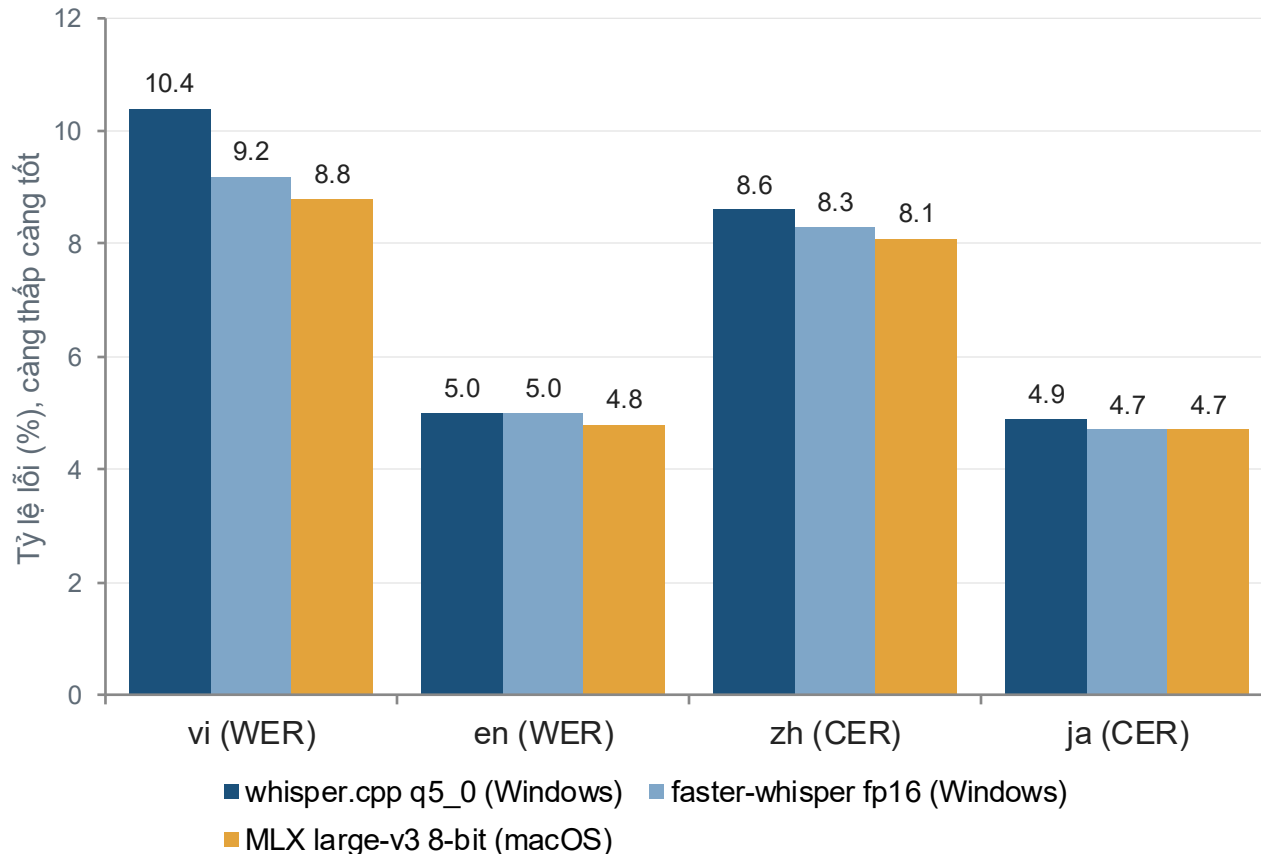
Dữ liệu	Dùng cho	Quy mô
FLEURS test [3], âm thanh	Nhận dạng (ASR)	3.099 bản thu, 10,22 giờ: vi 857 · en 647 · zh 945 · ja 650
FLEURS test, văn bản	Dịch máy (MT)	2.022 cặp câu, sáu chiều
FLEURS test, âm thanh	Độ trễ cả chuỗi	50 mẫu × 6 chiều
Âm thanh theo nhịp thật	Chạy liên tục	60 phút, 877 câu
3 bản ghi YouTube	Tách câu	Bản tin đọc · phỏng vấn tự phát · TEDx
25 đoạn phỏng vấn	ASR + MT giọng thật	149 s tiếng nói, câu tham chiếu nghe và sửa tay



Đo trên hai máy, FLEURS là dữ liệu chính; mọi số liệu chạy lại được từ tệp kết quả gốc

III. Kết quả thực nghiệm

2. Nhận dạng tiếng nói trên FLEURS (3.099 bản thu)



- Tiếng Việt sai gần gấp đôi tiếng Anh: WER **8,8–10,4%** so với 4,8–5,0%. FLEURS là giọng đọc rõ, nên đây là mức lỗi thấp nhất có thể kỳ vọng.
- Cùng large-v3-turbo, bản q5_0 (whisper.cpp) sai nhiều hơn bản fp16 (faster-whisper) 1,2 điểm WER tiếng Việt, nhưng nhanh gấp đôi (RTF 0,018 so với 0,034).
- Tiếng Trung và tiếng Nhật dùng CER vì không tách từ bằng khoảng trắng, như bài báo FLEURS.
- Ba runtime không dùng chung tệp mô hình, nên ba cột là ba cấu hình khác nhau.

➡ Trên giọng đọc, tiếng Việt sai khoảng 1 từ trong 10 (WER 8,8–10,4%), gấp đôi tiếng Anh

III. Kết quả thực nghiệm

2. Dịch máy trên FLEURS (NLLB-200 distilled 600M)

Chiều	Số câu	spBLEU	chrF++	COMET
vi→en	347	35,79	57,10	0,853
en→vi	347	37,13	54,69	0,851
vi→zh	346	17,15	16,14*	0,773
zh→vi	346	22,33	42,77	0,821
vi→ja	318	10,96	19,88*	0,824
ja→vi	318	19,91	40,19	0,819

* chrF++ của đích zh/ja không so ngang được (không có khoảng trắng); COMET dùng Unbabel/wmt22-comet-da [21]

spBLEU và COMET xếp hạng khác nhau

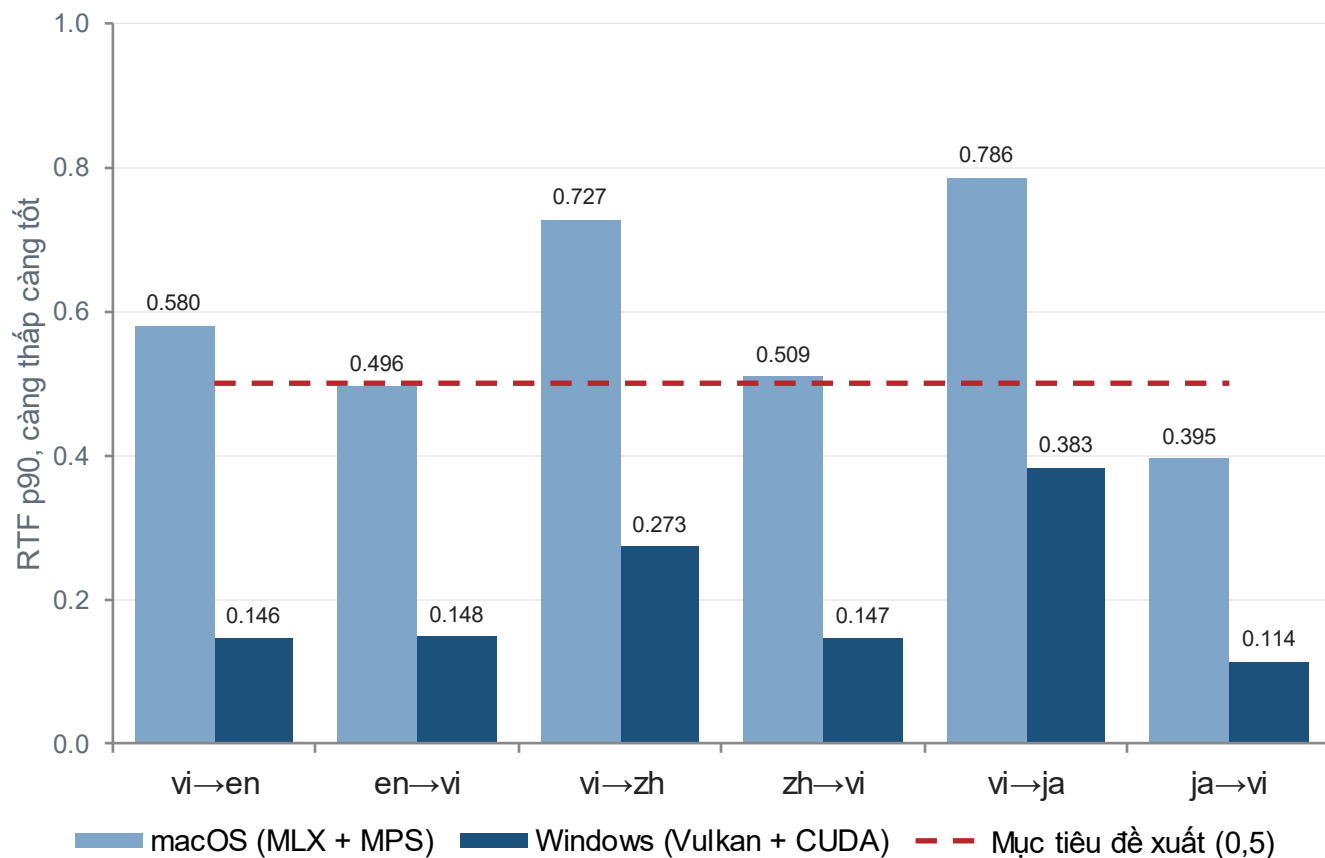
Xếp theo spBLEU thì vi→ja thấp nhất (10,96); xếp theo COMET thì vi→zh thấp nhất (0,773). spBLEU chỉ đếm chữ trùng với một bản dịch mẫu, nên câu đúng nghĩa mà diễn đạt khác vẫn bị điểm thấp; COMET chấm theo nghĩa.

Hai độ đo có thể xếp hạng ngược nhau, nên đồ án báo cáo cả hai. Chỗ chúng mâu thuẫn là chỗ phải đọc lại câu dịch thật.

➡ COMET 0,77–0,85 ở cả sáu chiều; yếu nhất là vi→zh, thấp ở cả spBLEU lẫn COMET

III. Kết quả thực nghiệm

3. Độ trễ: hệ số thời gian thực (RTF p90)



RTF = thời gian xử lý / thời lượng âm thanh
(50 mẫu mỗi chiều, đủ chuỗi VAD → ASR → MT → TTS)

- RTF p90 dưới 1 ở cả sáu chiều trên cả hai máy, tức hệ thống theo kịp luồng nói liên tục.
- Với ngưỡng 0,5, máy Windows đạt cả sáu chiều (xấu nhất vi→ja 0,383). Máy macOS đạt hai chiều en→vi và ja→vi; zh→vi trượt sát mép ở 0,509.
- Thời gian chờ chốt câu là 200–256 ms, chỉ phụ thuộc ngôn ngữ nguồn và giống nhau trên hai máy.

➡ Cả sáu chiều theo kịp người nói trên cả hai máy; máy có card đồ họa rời đạt luôn mục tiêu 0,5

III. Kết quả thực nghiệm

4. Nhận dạng và dịch trên hội thoại tự phát

WER tiếng Việt, cùng mô hình, cùng máy

10,4% → **24,6%**
FLEURS, 857 câu phòng vấn thật, 25 đoạn

Giọng nói tự phát cho WER gấp 2,4 lần giọng đọc

Lỗi chủ yếu: thuật ngữ tiếng Anh đọc theo giọng Việt

- "string" xuất hiện 18 lần, Whisper nghe đúng **1 lần**; nghe thành "stream" 6 lần, "trên" 5, "chuyên" 3.
- Phụ đề tự sinh của YouTube sai đúng kiểu đó: giới hạn chung của nhận dạng đa ngôn ngữ khi chen từ tiếng Anh vào câu tiếng Việt.

Hai lỗi khâu dịch chỉ lộ ra trên giọng thật

Vòng lặp mất kiểm soát

2/75 lượt, đều sang tiếng Nhật: câu mở đầu bằng từ đệm ("dạ đúng ạ...") thành "そうだ。" lặp hơn 50 lần. Chỉ có trên 256 token của NLLB chặn lại, và TTS sẽ đọc to toàn bộ.

Dịch nghĩa đen thuật ngữ

Đích tiếng Trung: "spring" thành 春天 (mùa xuân).
Đích tiếng Nhật: "spring" thành 春 trong "春のバッファー".
NLLB không có khái niệm thuật ngữ cần giữ nguyên.

Quy mô nhỏ (1 bản ghi, 25 đoạn), nên chỉ cho biết độ lớn của khoảng cách; số chính vẫn là các bảng FLEURS.

➡ Giọng nói tự nhiên có WER gấp 2,4 lần giọng đọc; lỗi chính là thuật ngữ tiếng Anh

III. Kết quả thực nghiệm

5. Đối chiếu với yêu cầu phi chức năng

Yêu cầu	Kết quả	Đánh giá
Phát hiện kết thúc câu < 700 ms	Chờ chót 200–256 ms (FLEURS), 0,20–0,36 s (giọng thật)	Đạt
ASR một câu ngắn < 1.500 ms	991–1.060 ms (macOS), 362–480 ms (Windows), câu 3 s	Đạt
Dịch < 1.000 ms; TTS < 1.500 ms	MT 460–579 ms; TTS 141–969 ms	Đạt
Tổng chiều đi < 4 s; phụ đề chiều về < 3 s	1,8–2,6 s; ASR + MT < 1,2 s trên Windows	Đạt
Chạy 60 phút, bộ nhớ không tăng đều	877 câu, 0 lỗi, +12 MB	Đạt
Một mô hình lỗi không làm mất cả phiên	Câu lỗi chuyển Error, phiên chạy tiếp (kiểm thử đơn vị)	Đạt
Offline sau khi cài mô hình	Không có lời gọi mạng trên đường dịch; tài liệu API đóng gói sẵn	Đạt
Không gửi, không lưu âm thanh; tắt được lịch sử	Âm thanh chỉ qua 127.0.0.1; lịch sử chỉ lưu văn bản	Đạt
Windows 11 và macOS Apple Silicon	Có bộ cài và số đo trên cả hai	Đạt
Đổi runtime không sửa chuỗi xử lý	Thêm MLX, faster-whisper không sửa dòng nào	Đạt

 **Đạt cả 10 yêu cầu; riêng chạy offline mới đạt theo thiết kế, chưa có bài thử ngắt mạng**

IV. Kết luận

1. Kết quả đạt được



Chuỗi xử lý bốn khâu, thay được từng khâu

6 chiều dịch Việt ↔ Anh / Nhật / Trung; 3 runtime ASR thay thế nhau, thêm vào không sửa chuỗi xử lý.



Hai nguồn âm thanh, không vòng lặp

Micro và âm thanh hệ thống thu đồng thời, tách bạch; bản dịch đang phát không bị thu ngược.



Chạy trên hai nền tảng

Bộ cài tự chứa cho Windows 11 và macOS Apple Silicon; whisper.cpp chạy GPU qua Vulkan với +18 MB.



Đánh giá bằng độ đo chuẩn trên FLEURS

WER vi 8,8–10,4%, en 4,8–5,0%; CER zh/ja < 9%; COMET 0,77–0,85; Windows: cả 6 chiều RTF p90 ≤ 0,5; 60 phút không lỗi.



Đo cả trên giọng thật

Phòng vấn thật: WER vi 24,6% so với 10,4% trên FLEURS; bộ tách câu đưa câu p90 của bản tin từ 12,5 s xuống 5,9 s.



Ứng dụng dịch hai chiều chạy trọn trên laptop, đo từng khâu trên FLEURS và trên giọng thật

IV. Kết luận

2. Hạn chế và hướng phát triển

Hạn chế

Thuật ngữ tiếng Anh trong câu tiếng Việt: "string" đúng 1/18 lần

NLLB rơi vào vòng lặp (2/75 lượt), dịch nghĩa đen thuật ngữ

TTS đích ja/zh chậm, chạy CPU, ~2/3 thời gian chuỗi

Đo giọng thật quy mô nhỏ; chưa đo trên máy chỉ có CPU

NLLB-200 giấy phép phi thương mại (CC-BY-NC-4.0)

Hướng phát triển

Danh sách thuật ngữ: gợi ý cho Whisper (initial_prompt) và giữ nguyên khi dịch

Chặn lặp bằng tham số sinh hoặc bộ lọc sau dịch, rồi chạy lại bộ đánh giá MT

Đưa TTS lên GPU cho hai đích tiếng Nhật và tiếng Trung

Nhiều bản ghi, nhiều chủ đề, hai người nghe độc lập; thêm cấu hình máy yếu

Thử mô hình dịch khác trên cùng bộ đánh giá, như đã làm với ASR

➡ Hạn chế lớn nhất là thuật ngữ tiếng Anh trong câu tiếng Việt; bước tiếp theo là danh sách thuật ngữ

DEMO

Nói tiếng Việt, nghe bản dịch tiếng Anh
Phụ đề cho video tiếng Anh · Tất mạng vẫn chạy

Q&A

The background features a central white area with organic, wavy borders in shades of light blue and dark blue. Scattered throughout are small, dried, brownish-gold leaves and petals, some of which are also arranged in larger, more structured clusters on the left and right sides.

**Cảm ơn quý Thầy Cô
đã lắng nghe**

Slide dự phòng

Mục lục: gõ số trang rồi Enter để nhảy tới slide đó

Thuật ngữ

- 28** VAD: phát hiện giọng nói
- 29** ASR: nhận dạng tiếng nói
- 30** MT: dịch máy
- 31** TTS: tổng hợp tiếng nói
- 32** Thuật ngữ kỹ thuật khác

Dữ liệu và độ đo

- 33** Bộ dữ liệu FLEURS
- 34** WER và CER
- 35** spBLEU và chrF++
- 36** COMET
- 37** RTF và p90

Câu hỏi và chi tiết

- 38** Thêm ngôn ngữ khác
- 39** Mô hình end-to-end, LLM
- 40** Lựa chọn mô hình, preset
- 41** Nạp mô hình, GPU Windows
- 42** Điểm nghe TTS
- 43** Câu 3 s và chạy 60 phút
- 44** Tách câu trên giọng thật
- 45** Tài liệu tham khảo

Slide dự phòng

VAD (Voice Activity Detection): phát hiện giọng nói

Dòng âm thanh vào bộ tách câu (minh họa)



Là gì

Phân loại từng khung âm thanh ngắn: có người đang nói hay không. VAD không nhận dạng nội dung.

Vào: PCM 16 kHz, mỗi khung 32 ms.

Ra: xác suất có giọng nói (0 đến 1).

Trong đồ án

Silero VAD chạy trên CPU. Bộ tách câu gom các khung có giọng thành một câu và chốt câu khi:

- im lặng 320 ms với câu ngắn
- hụt hơi 140 ms khi câu > 3,5 s
- chạm trần 6 s (preset Cân bằng)

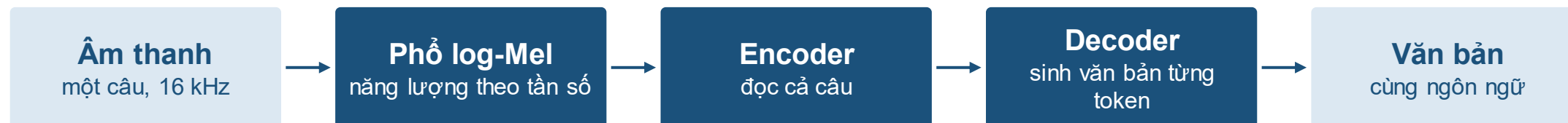
Vì sao cần

Whisper cần nhận trọn một câu. Đưa đoạn im lặng vào thì Whisper hay bịa ra câu kiểu "Cảm ơn đã xem video".

VAD bỏ phần im lặng và quyết định lúc nào gửi câu đi dịch.

Slide dự phòng

ASR (Automatic Speech Recognition): nhận dạng tiếng nói



Whisper [19]: mô hình Transformer encoder-decoder của OpenAI, nhận dạng khoảng 100 ngôn ngữ

Là gì

Chuyển giọng nói thành văn bản cùng ngôn ngữ. Đây là khâu thứ hai, nhận câu do VAD cắt ra.

Một bộ lọc sau ASR bỏ những câu Whisper bịa ra khi gặp tiếng ồn.

Trong đề án

Preset Cân bằng dùng large-v3-turbo lượng tử 5 bit (q5_0). Bản turbo giảm decoder từ 32 xuống 4 lớp nên nhanh hơn nhiều so với large-v3.

Ba runtime theo phần cứng: whisper.cpp (Vulkan, Metal), faster-whisper (CUDA), MLX (Apple).

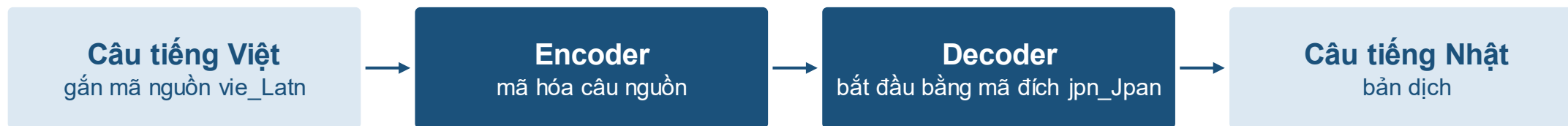
Đo bằng

WER cho tiếng Việt, tiếng Anh; CER cho tiếng Trung, tiếng Nhật.

3.099 bản thu FLEURS: WER vi 8,8–10,4%, en 4,8–5,0%; CER zh, ja dưới 9%.

Slide dự phòng

MT (Machine Translation): dịch máy



Mã ngôn ngữ đích quyết định decoder sinh ra tiếng nào, nên Việt sang Nhật dịch thẳng, không qua tiếng Anh

Là gì

Dịch văn bản từ ngôn ngữ nguồn sang ngôn ngữ đích. Khâu này nhận văn bản do ASR chép ra.

Mã dùng trong đồ án: vie_Latn, eng_Latn, jpn_Jpan, zho_Hans.

NLLB-200

Mô hình dịch của Meta (2022): một mô hình cho 200 ngôn ngữ [4].

Bản distilled 600M (600 triệu tham số) được chưng cất từ mô hình 54,5 tỷ tham số, đủ nhẹ để chạy trên laptop. Giấy phép CC-BY-NC-4.0.

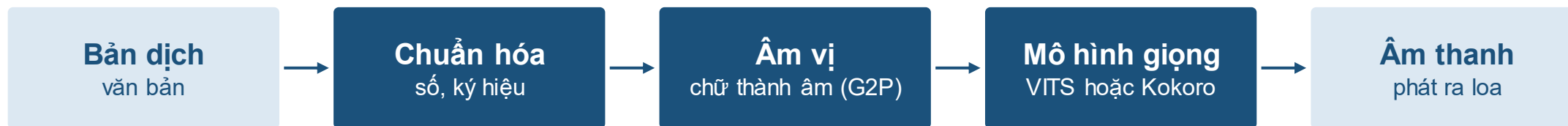
Trong đồ án và cách đo

Chạy PyTorch trên CUDA (Windows) hoặc MPS (Mac).

Đo trên văn bản FLEURS, không qua ASR: spBLEU, chrF++ và COMET. COMET 0,77–0,85 ở sáu chiều.

Slide dự phòng

TTS (Text-to-Speech): tổng hợp tiếng nói



Các bước chung của một hệ TTS

Là gì

Đọc văn bản thành giọng nói. Chỉ chạy ở chiều micro: đọc bản dịch câu người dùng vừa nói, ra loa cho người đối diện nghe.

Chiều âm thanh hệ thống chỉ hiện phụ đề, để ứng dụng không thu lại giọng đọc của chính nó.

Trong đề án

sherpa-onnx chạy giọng VITS cho tiếng Việt, Anh, Trung, hoàn toàn offline.

Tiếng Nhật: sherpa-onnx không xử lý được chữ Nhật, nên em dùng Kokoro; OpenJTalk chuyển chữ Nhật thành âm vị. LanguageRoutedTts chọn engine theo ngôn ngữ đích.

Giới hạn

TTS chạy trên CPU.

Trên Windows, sau khi ASR và MT lên GPU, TTS chiếm 63% thời gian cả chuỗi với đích tiếng Trung và 65% với đích tiếng Nhật.

Slide dự phòng

Các thuật ngữ kỹ thuật khác trong bài

Thuật ngữ	Nghĩa trong đề án
PCM 16-bit, 16 kHz, một kênh	Âm thanh số không nén, 16.000 mẫu mỗi giây; định dạng chung trước khi vào VAD
REST và WebSocket	REST cho cấu hình, mô hình, lịch sử. WebSocket giữ kết nối mở: gửi âm thanh lên và nhận kết quả về liên tục
Ports & Adapters	Chuỗi xử lý chỉ gọi giao diện trừu tượng (port); mỗi mô hình cụ thể là một adapter cắm vào
Runtime	Thư viện chạy mô hình: whisper.cpp, faster-whisper, MLX, PyTorch, ONNX Runtime
Lượng tử hóa (q5_0, 8-bit)	Lưu trọng số bằng ít bit hơn: mô hình nhỏ và nhanh hơn, độ chính xác giảm một chút
Vulkan, CUDA, Metal, MPS	Cách chạy tính toán trên GPU: Vulkan có trong driver mọi hãng, CUDA của NVIDIA, Metal và MPS của Apple
p50, p90	50% hoặc 90% số câu nhanh hơn mức này; p90 lộ ra các câu chậm mà trung bình che mất
Preset	Bộ cấu hình đặt sẵn Nhanh, Cân bằng, Chất lượng: chọn mô hình ASR và trần độ dài câu

Slide dự phòng

Bộ dữ liệu FLEURS: đánh giá trên những gì?

Ngôn ngữ	Bản thu (ASR)	Giờ âm thanh	TB mỗi bản thu	Cặp câu với vi (MT)
vi	857	3,0	12,6 s	không áp dụng
en	647	1,8	9,9 s	347
zh	945	3,1	11,7 s	346
ja	650	2,4	13,1 s	318
Tổng	3.099	10,2		2.022

Dùng vào đâu

- ASR: âm thanh và bản chép lời đã chuẩn hóa sẵn, chấm WER/CER trên 3.099 bản thu.
- MT: ghép hai ngôn ngữ theo mã câu, được 2.022 cặp câu sáu chiều (mỗi cặp dùng cho hai chiều); dịch văn bản, không qua ASR.
- Độ trễ: 50 bản thu mỗi chiều, chạy đủ chuỗi VAD → ASR → MT → TTS.
- Chỉ dùng tập test; em không huấn luyện gì trên FLEURS.

Nguồn gốc

Google, 2022 [3]; 102 ngôn ngữ. Giấy phép CC-BY-4.0.

Câu lấy từ FLoRes-101: câu tiếng Anh trên Wikinews, Wikijunior, Wikivoyage, được dịch chuyên nghiệp sang từng ngôn ngữ rồi người bản ngữ đọc to.

Câu dài trung bình 31 từ (vi); giọng đọc rõ, ít nhiễu.

Ví dụ: một mã câu ở bốn ngôn ngữ

VI Bản tin truyền hình cho thấy có khói trắng bốc ra từ nhà máy.

EN Television reports show white smoke coming from the plant.

ZH 电视报道显示, 有白烟从工厂里冒出。

JA テレビの報道によると、原発から白煙が上がっています。

Bản tiếng Nhật ghi "nhà máy điện hạt nhân" (原発): bốn bản đều dịch từ câu tiếng Anh, không dịch từ nhau.

Slide dự phòng

WER và CER: đo lỗi nhận dạng

$$\text{WER} = (\text{S} + \text{D} + \text{I}) / \text{N}$$

S: thay, từ bị nghe nhầm

D: xóa, từ bị bỏ sót

I: chèn, từ máy tự thêm

N: số từ của câu tham chiếu

Càng thấp càng tốt. I không bị chặn nên WER có thể vượt 100%.

CER cho tiếng Trung, tiếng Nhật

Cùng công thức, nhưng đếm trên ký tự thay vì từ.

Hai thứ tiếng này không tách từ bằng khoảng trắng, nên WER chỉ chấm theo chỗ Whisper tình cờ chèn dấu cách. Bài báo FLEURS và Whisper cũng dùng CER.

WER và CER không so với nhau được.

Ví dụ tính tay (một lỗi bắt được trên máy thật)

Tham chiếu	độ	trễ	hiện	tại	là	hai	giây
Máy nghe	vụ	trễ	hiện	tại	(mất)	hai	giây
Loại	S	C	C	C	D	C	C

$$\text{S} = 1, \text{D} = 1, \text{I} = 0, \text{N} = 7$$
$$\text{WER} = 2/7 = 28,6\%$$

Trên cả tập, em cộng lỗi của mọi câu rồi chia cho tổng số từ (WER gộp). Lấy trung bình từng câu thì một câu 3 từ sai hết sẽ nặng ngang một câu 40 từ.

Slide dự phòng

spBLEU và chrF++: đo độ trùng với bản dịch tham chiếu

BLEU

$$\text{BLEU} = \text{BP} \times \exp\left(\frac{1}{4} \times \sum \log p_n\right), \quad n = 1 \dots 4$$

- p_n : tỉ lệ cụm n từ của bản dịch máy có trong bản tham chiếu ($n = 1$ đến 4).
- BP: phạt bản dịch ngắn hơn tham chiếu; $\text{BP} = e^{(1 - r/c)}$ khi độ dài máy dịch $c \leq$ độ dài tham chiếu r .
- Thang 0 đến 100, càng cao càng tốt; báo cáo cho cả tập.

spBLEU và chrF++

spBLEU: tách từ bằng SentencePiece dùng chung cho mọi ngôn ngữ (tokenizer flores200), nên chấm được tiếng Trung, Nhật và so được với số công bố của NLLB-200.

chrF++: F-score trên cụm ký tự (tối 6 ký tự) cộng cụm từ (tối 2 từ), ổn định hơn BLEU trên tập vài trăm câu. Đích zh, ja không có khoảng trắng nên phần cụm từ gần bằng 0: hai ô đó có dấu *.

Ví dụ: vì sao BLEU khắc nghiệt trên một câu

Tham chiếu	I do not agree with this proposal.
Máy dịch	I disagree with this proposal.

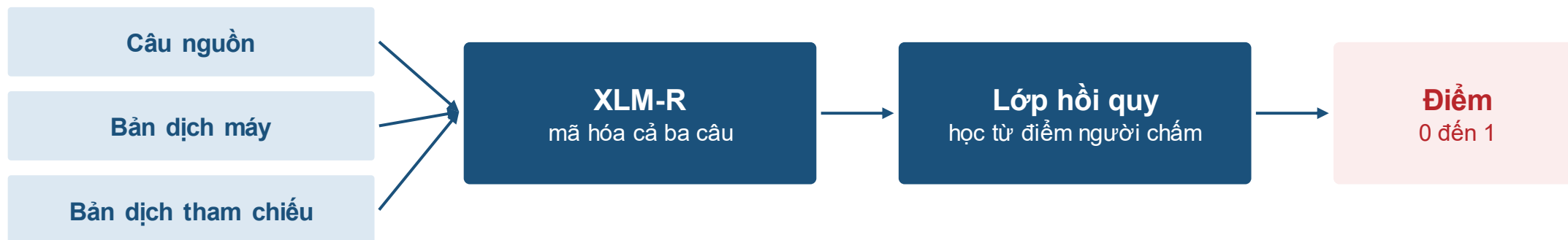
p_1	p_2	p_3	p_4	BP	BLEU
4/5	2/4	1/3	0/2	0,67	0

Không có cụm 4 từ nào trùng nên BLEU của câu bằng 0, dù câu dịch đúng nghĩa. Vì vậy BLEU chỉ báo cáo cho cả tập, và đề tài dùng thêm COMET.

Không so spBLEU giữa hai chiều dịch khác nhau: mỗi chiều khó một kiểu.

Slide dự phòng

COMET: chấm bản dịch theo nghĩa



Cách hoạt động

Nền là XLM-R, một mô hình ngôn ngữ đa ngữ, được huấn luyện thêm trên điểm người thật chấm bản dịch máy (Direct Assessment của WMT).

Checkpoint: Unbabel/wmt22-comet-da [21], như thầy giao.

Đọc điểm thế nào

Thang 0 đến 1, càng cao càng tốt.

0,85 không có nghĩa là dịch đúng 85%. Điểm dùng để xếp hạng các chiều hoặc các mô hình trên cùng tập test, với cùng checkpoint.

Vì sao dùng thêm COMET

Nguồn: "Tôi không đồng ý với đề xuất này."

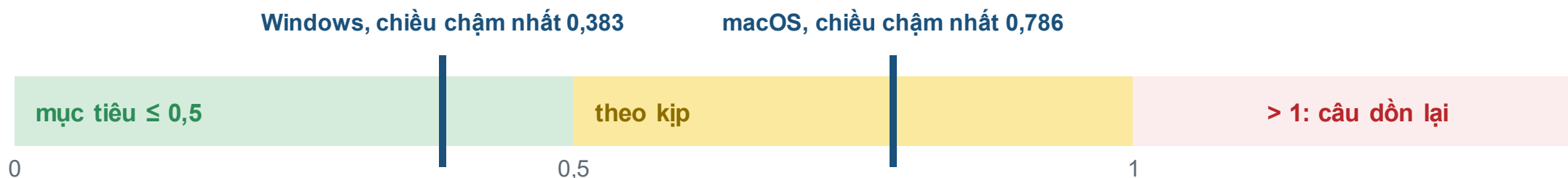
"I disagree with this proposal." đúng nghĩa nhưng BLEU câu bằng 0. "I do not agree with this purpose." trùng nhiều chữ nhưng sai nghĩa.

COMET nhìn cả câu nguồn nên chấm theo nghĩa.

RTF và p90: đo tốc độ cả chuỗi

$$\text{RTF} = \text{thời gian xử lý} \div \text{thời lượng âm thanh}$$

Ví dụ: câu dài 4 giây, cả chuỗi VAD → ASR → MT → TTS xử lý mất 1,2 giây → $\text{RTF} = 0,3$



Vì sao phải dưới 1

RTF lớn hơn 1 thì một câu xử lý lâu hơn thời gian nói câu đó: người nói liên tục làm các câu dồn lại, độ trễ tăng mãi. Các bài về ASR trực tuyến lấy $\text{RTF} \leq 1$ làm điều kiện thời gian thực.

Vì sao mục tiêu 0,5

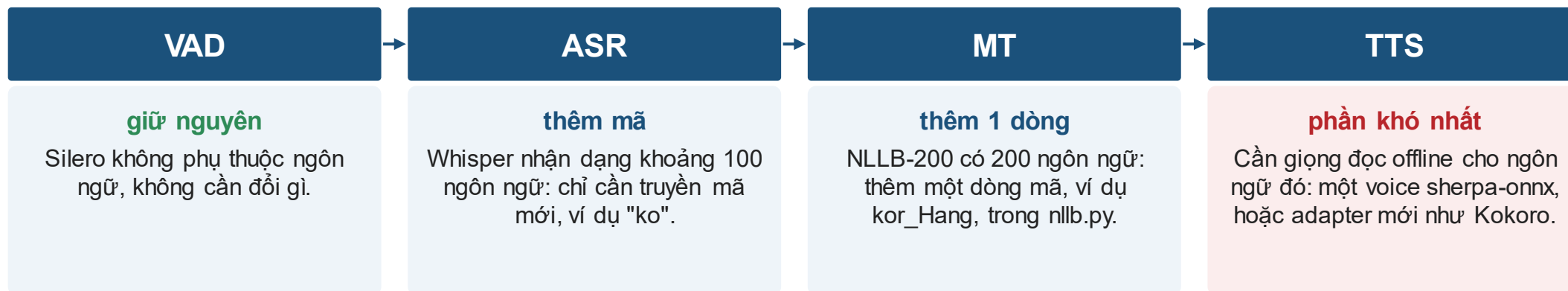
Mức em đề xuất. Hai chiều dùng chung một bộ mô hình, khi hai bên cùng nói mỗi chiều còn khoảng nửa sức máy. Câu dài nhất $6 \text{ s} \times 0,5 = 3 \text{ s}$ xử lý, khớp mức trễ khoảng 3 s của phiên dịch viên người thật.

Vì sao dùng p90

p90: 9 trên 10 câu nhanh hơn mức này. Trung bình có thể thấp trong khi vài câu dài rất chậm. Thời gian VAD chờ chốt câu (200–256 ms) tính riêng, không nằm trong RTF.

Slide dự phòng

Câu hỏi: muốn thêm ngôn ngữ khác thì phải sửa những gì?



Các chỗ phải sửa

- Thêm mã vào enum Language (Python) và bản sao TypeScript.
- Thêm mã NLLB vào adapters/mt/nllb.py.
- Thêm giọng đọc; LanguageRoutedTts chọn engine theo ngôn ngữ.
- Tên ngôn ngữ trên giao diện; ngôn ngữ không có khoảng trắng thì chấm bằng CER.

Giữ nguyên: pipeline, VAD, bộ tách câu, WebSocket.

Ảnh hưởng tới chất lượng

Chất lượng do Whisper và NLLB quyết định, và khác nhau nhiều giữa các ngôn ngữ: ngôn ngữ ít dữ liệu huấn luyện thì WER cao hơn hẳn.

FLEURS có 102 ngôn ngữ, nên chạy lại đúng bộ đánh giá của đề tài để có WER, COMET, RTF cho ngôn ngữ mới trước khi đưa vào ứng dụng.

➡ Thêm ngôn ngữ chỉ dựa tới cấu hình và giọng đọc; còn chất lượng thì phải đo lại trên FLEURS

Slide dự phòng

Câu hỏi: sao không dùng một mô hình end-to-end hoặc LLM?

Một mô hình làm trọn (SeamlessM4T, LLM)

- Ít lỗi cộng dồn giữa các khâu.
- LLM 7 tỷ tham số lượng tử 4 bit: khoảng 3,5 GB chỉ riêng trọng số, chưa tính phần nghe và đọc.
- Kết quả sai thì không biết sai ở phần nghe hay phần dịch, và không thay riêng được phần nào.

2,3 tỷ

tham số trong một khối: SeamlessM4T v2 large

Chuỗi bốn khâu của đề tài

- Mỗi khâu một mô hình vừa phải, thay được độc lập.
- Mỗi khâu chạy runtime nhanh nhất cho phần cứng: Vulkan, CUDA, Metal.
- Đo được lỗi từng khâu; trên laptop RTX 4060, RTF p90 từ 0,11 đến 0,38.
- Cái giá là lỗi và độ trễ cộng dồn, đã đo ở phần III.

1,4 tỷ

tham số, chia ở hai khâu: Whisper turbo 0,8 + NLLB 0,6



Mô hình một bước lớn hơn và không đo, thay được từng phần; nên đề tài ghép các mô hình vừa phải

Slide dự phòng

Lựa chọn mô hình và preset (bổ sung phần II)

Khâu	Mô hình / runtime	Lý do chọn
VAD	Silero VAD [24]	~2 MB, chạy CPU, mỗi khung 32 ms xử lý dưới 1 ms
ASR	Whisper large-v3 / large-v3-turbo [19] trên 3 runtime: whisper.cpp (Vulkan, Metal), faster-whisper (CUDA), MLX (Apple Silicon)	Đa ngôn ngữ sẵn; có bản lượng tử hóa chạy được trên máy cá nhân; mỗi nền tảng một runtime nhanh nhất
MT	NLLB-200 distilled 600M [4]	Một mô hình phủ đủ 6 chiều, dịch trực tiếp Việt ↔ Nhật / Trung không qua tiếng Anh
TTS	sherpa-onnx [12] (vi, en, zh) + Kokoro [9] với OpenJTalk (ja)	Chạy ONNX offline; sherpa-onnx không có front-end tiếng Nhật dùng được nên ghép thêm Kokoro

Ba mức cấu hình sẵn, đổi được ngay trong ứng dụng:

Nhanh

whisper small · trần câu 4,5 s

Cân bằng

large-v3-turbo q5 · trần câu 6 s

Chất lượng

large-v3 · trần câu 8 s

Slide dự phòng

Nạp mô hình theo yêu cầu, GPU trên Windows (bổ sung phần II)

45 s $\rightarrow$ 0,4 s

Thời gian mở dịch vụ

Không nạp gì lúc khởi động; mọi đường cần mô hình đi qua một cửa `ModelManager.ensure_loaded()`. Chọn preset không tải gì.

17,6 s $\rightarrow$ 0,44 s

ASR cho 3 s âm thanh trên bản cài Windows

`whisper.cpp` chạy GPU qua Vulkan thay vì CPU.
17.573 ms $\rightarrow$ 437 ms trên RTX 4060.

+18 MB

thay vì ~1 GB nếu đóng gói CUDA

Vulkan có sẵn trong driver NVIDIA, AMD, Intel: một bộ cài chạy GPU trên mọi hãng. Ý tưởng từ TranscriptionSuite [10].

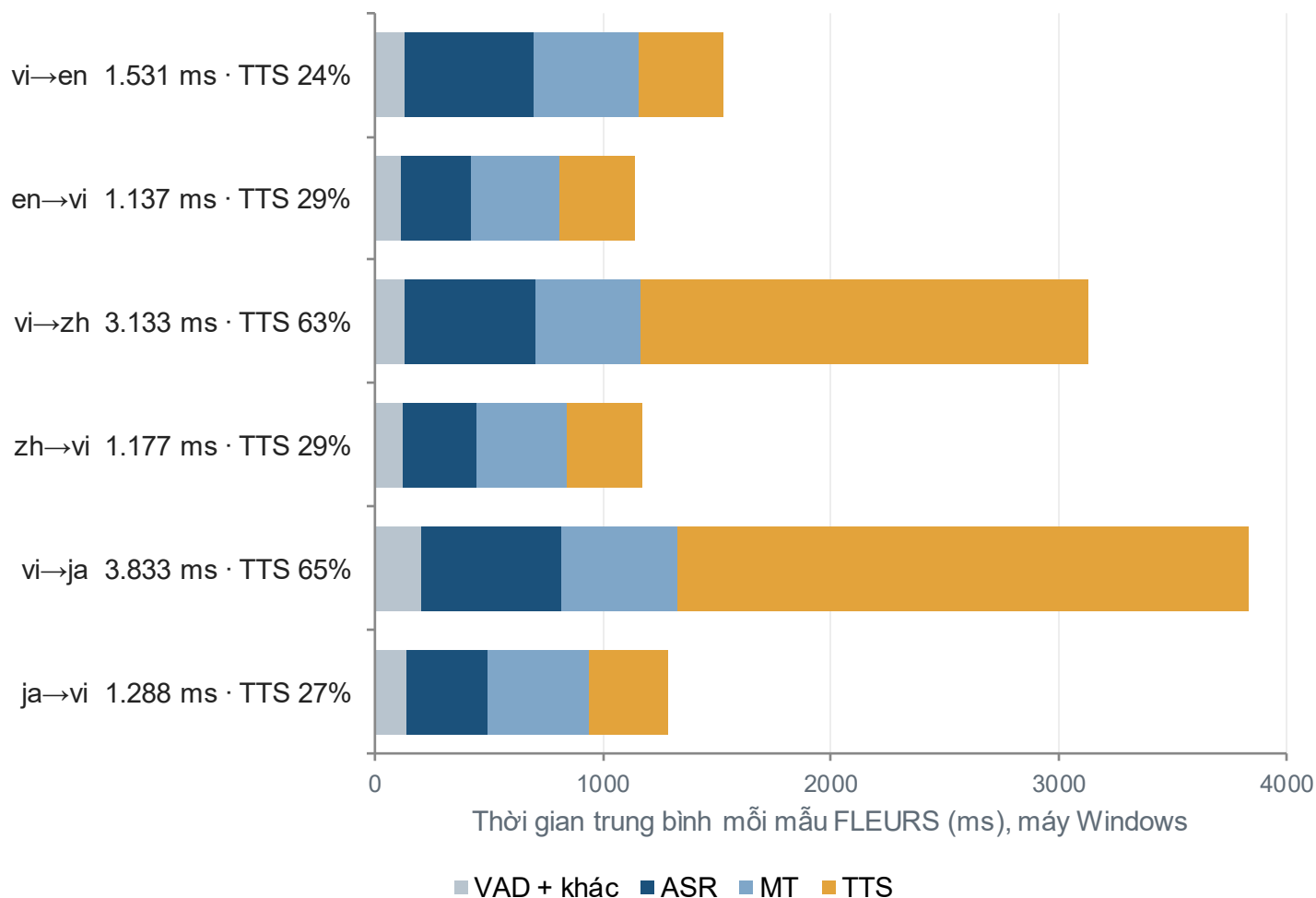
9,9 s $\leftrightarrow$ 0,17 s

GPU tích hợp so với GPU rời

`whisper.cpp` lấy GPU đầu tiên, trên laptop hai card là Iris Xe. Adapter tự liệt kê thiết bị và chọn card rời.

Slide dự phòng

Điểm nghẽn chuyển sang TTS khi có GPU (bổ sung phần III)



- So với máy macOS, GPU làm ASR nhanh khoảng 8 lần (whisper.cpp/Vulkan) và MT nhanh 2 đến 2,5 lần (NLLB/CUDA).
- TTS vẫn chạy CPU nên gần như giữ nguyên, và thành khâu chậm nhất ở hai đích zh/ja: **63% và 65%** toàn chuỗi.
- **Chỗ cần tối ưu tiếp theo là TTS cho đích tiếng Nhật và tiếng Trung.**

Slide dự phòng

Câu 3 giây và chạy liên tục 60 phút (bổ sung phần III)

Một câu 3 giây, máy macOS, preset Cân bằng

Chiều	VAD	ASR	MT	TTS	Tổng
vi→en	46	1.060	569	141	1.819
en→vi	46	991	579	148	1.766
vi→ja	48	1.056	460	772	2.338
vi→zh	46	1.038	558	969	2.613
Mục tiêu		< 1.500	< 1.000	< 1.500	< 4.000

Đơn vị: ms. Cả bốn mốc đều đạt ở cả bốn chiều. Trên bản cài Windows, cùng phép đo cho ASR 362–480 ms.

Chạy liên tục 60 phút (Windows)

877 / 0

câu thành công / lỗi

+12 MB

bộ nhớ RSS (1.110 → 1.122 MB)

+1,8 ms

độ trễ 10% câu cuối so với 10% câu đầu

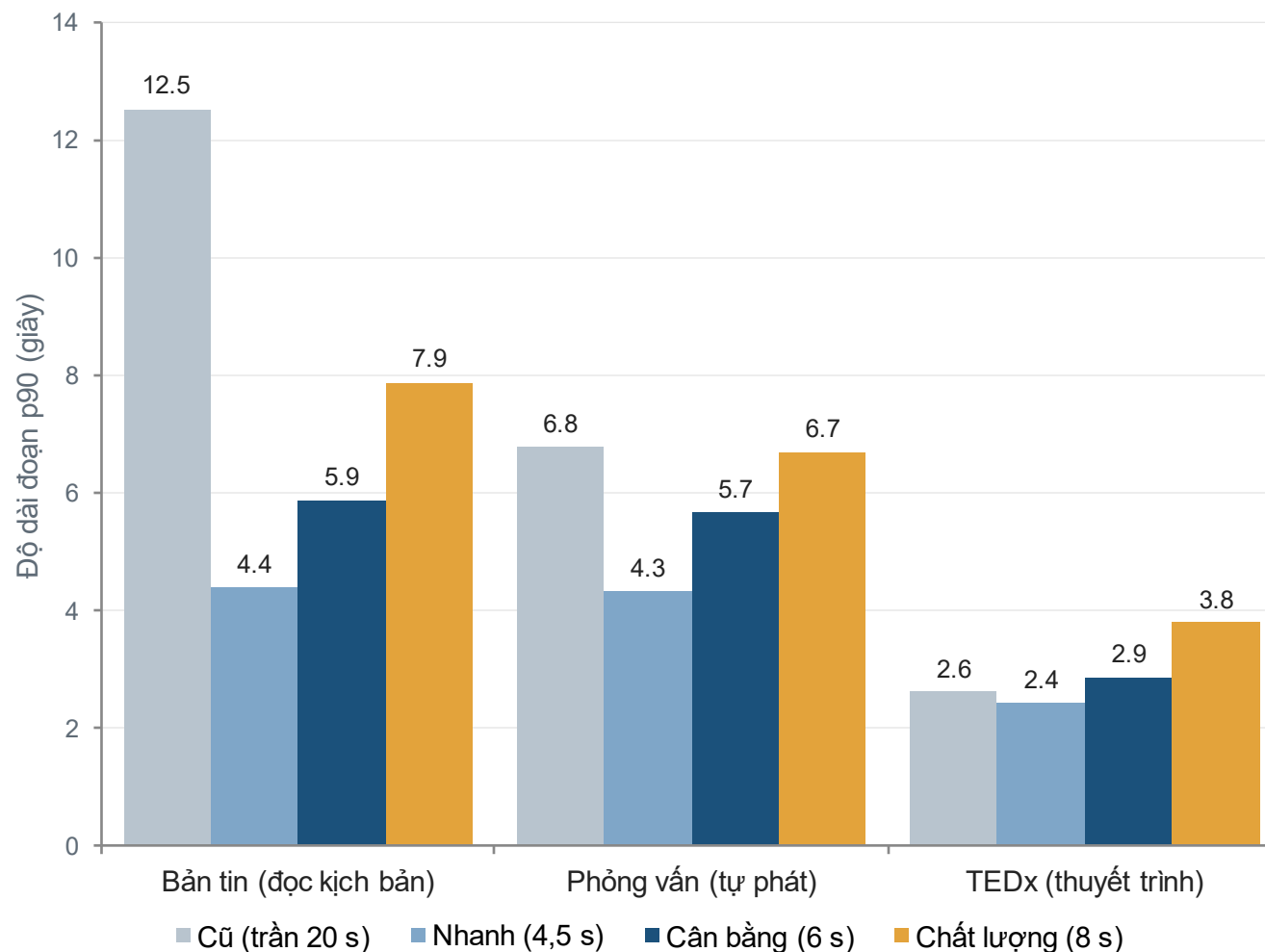
443 ms

độ trễ p50 (p95 541 ms), không mất kết nối

Sau một giờ, bộ nhớ và độ trễ gần như đứng yên: không có dấu hiệu rò rỉ đáng kể.

Slide dự phòng

Tách câu trên ba loại giọng người thật (bổ sung phần III)



- Với tham số cũ, 10% số câu của bản tin dài hơn 12,5 s, đúng chỗ GVHD góp ý. Preset Cân bằng còn 5,9 s.
- Thời gian chờ chốt gần như không đổi (0,20–0,36 s), nên phần cải thiện đến từ trần độ dài.
- Cái giá là cắt giữa câu: 30% số đoạn bản tin, 10% ở phỏng vấn, 0% ở TEDx. Giọng nói tự nhiên có nhiều chỗ ngừng nên ít bị cắt hơn giọng đọc.

- [2] A. Cockburn, "**Hexagonal architecture**," 2005. <https://alistair.cockburn.us/hexagonal-architecture/>
- [3] A. Conneau et al., "**FLEURS: Few-shot learning evaluation of universal representations of speech**," in Proc. IEEE SLT, 2022.
- [4] M. R. Costa-jussà et al. (NLLB Team), "**No language left behind: Scaling human-centered machine translation**," arXiv:2207.04672, 2022.
- [7] Google, "**Speech translation in Google Meet**," Google I/O, May 2025.
- [9] hexgrad, "**Kokoro-82M**," Hugging Face, 2025.
- [10] homelab-00, "**TranscriptionSuite: A fully local and private speech-to-text application**," GitHub, 2026.
- [12] k2-fsa, "**sherpa-onnx: Speech-to-text, text-to-speech and audio processing with ONNX Runtime**," 2026.
- [19] A. Radford et al., "**Robust speech recognition via large-scale weak supervision**," in Proc. ICML, 2023, pp. 28492–28518.
- [21] R. Rei et al., "**COMET-22: Unbabel-IST 2022 submission for the metrics shared task**," in Proc. WMT, 2022, pp. 578–585.
- [22] royshil, "**LocalVocal: Local live captions & translation on-the-go (OBS plugin)**," GitHub.
- [23] Seamless Communication et al., "**SeamlessM4T: Massively multilingual & multimodal machine translation**," arXiv:2308.11596, 2023.
- [24] Silero Team, "**Silero VAD: Pre-trained enterprise-grade voice activity detector**," GitHub, 2026.

Số thứ tự theo danh mục tài liệu tham khảo của quyển đồ án (25 tài liệu).